

2026 SISBID Unsupervised Learning Comprehensive Lab

Genevera I. Allen & Yufeng Liu

Data description

This lab uses TCGA BRCA RNA-sequencing and clinical data.

Gene expression data: **gdat**

- **gdat** contains gene expression measurements for 445 patients and 353 genes.
- The retained genes are somatic-mutation-associated genes from COSMIC.
- The expression data are Level III TCGA BRCA RNA-sequencing data.
- The data have already been preprocessed:
 - reads normalized by RPKM;
 - overdispersion corrected by a log-transformation, $\log(1 + \text{data})$;
 - short gene names are provided as column names.

Clinical data: **cdat**

- **cdat** contains clinical information for the same 445 patients.
- Clinical variables include:
 - **Subtype**: PAM50 subtype, including Basal-like, Luminal A, Luminal B, HER2-enriched, and Normal-like;
 - **ER**: estrogen-receptor status;
 - **PR**: progesterone-receptor status;
 - **HER2**: human epidermal growth factor receptor 2 status;
 - **Node**: number of lymph nodes involved;
 - **Metastasis**: indicator for whether the cancer has metastasized.

Lab goals

This is a comprehensive exploratory data analysis lab. Your goal is not to run every possible method, but to use multiple unsupervised-learning and statistical tools to develop a coherent understanding of the data.

For each method you use, think about:

1. What structure is the method designed to reveal?
2. What preprocessing choices does the method require?
3. How stable are the results?
4. How do the results relate to the clinical variables?
5. What biological or statistical conclusion is supported by the evidence?

Problems

Problem 1: Dimension reduction

- 1a. Apply several dimension-reduction methods to this dataset. At minimum, compare PCA and MDS. If time permits, also explore UMAP and t-SNE.
- 1b. Relate the dimension-reduction results to the clinical data. Is any clinical information reflected in the lower-dimensional spaces?
- 1c. Overall, which dimension-reduction method do you recommend for this dataset, and why?

Problem 2: Clustering

- 2a. Apply several clustering algorithms, such as K-means, hierarchical clustering, spectral clustering, or biclustering. Explore different values of K, distance metrics, or linkage functions where appropriate.
- 2b. Relate the clustering results to the clinical data. Can any clustering method recover clinical information such as PAM50 subtype?
- 2c. Optional: assess the stability or validation of one clustering result.
- 2d. Overall, which clustering method or methods do you recommend for this dataset, and why?

Problem 3: Multiple comparisons

- 3a. Identify genes associated with ER status, PR status, HER2 status, and metastasis status.
- 3b. Compare different multiple-comparisons procedures, such as Bonferroni, Holm, BH/FDR, and BY.
- 3c. Compare the gene lists identified under different procedures and clinical responses. Which approach is most appropriate for this exploratory analysis, and why?

Problem 4: Graphical models

- 4a. Use graphical models to explore relationships among genes.
- 4b. Are any well-connected genes related to patterns found in the dimension-reduction, clustering, or testing analyses?
- 4c. How sensitive are the estimated networks to the tuning parameter or method?

Problem 5: Visualization

- 5a. Visualize this dataset using multiple approaches.
- 5b. Prepare the best visual summary of this dataset for a short presentation.

Problem 6: Exploratory data analysis summary

- 6a. What is the most interesting finding from your analysis?
- 6b. Is this finding consistent and stable across methods?
- 6c. Prepare a visual summary that best illustrates your most interesting finding.

Starter R code

The code below is designed to help you begin the lab. It is intentionally incomplete: you should modify, extend, and interpret the code to answer the questions above.

Setup

```
knitr::opts_chunk$set(  
  echo = TRUE,  
  message = FALSE,  
  warning = FALSE,  
  fig.width = 7,  
  fig.height = 5  
)  
  
set.seed(2026)  
  
pkg_available <- function(pkg) {  
  requireNamespace(pkg, quietly = TRUE)  
}  
  
load_pkg <- function(pkg) {  
  if (pkg_available(pkg)) {  
    suppressPackageStartupMessages(library(pkg, character.only = TRUE))  
    TRUE  
  } else {  
    message("Package '", pkg, "' is not installed; skipping code that requires it.")  
    FALSE  
  }  
}  
  
load_pkg("ggplot2")  
  
## [1] TRUE  
load_pkg("GGally")  
  
## [1] TRUE  
load_pkg("igraph")  
  
## [1] TRUE  
load_pkg("umap")  
  
## [1] TRUE  
load_pkg("Rtsne")  
  
## [1] TRUE
```

Load and inspect data

```
load("UnsupL_SISBID_2026.Rdata")  
  
# Basic checks  
stopifnot(exists("gdat"), exists("cdat"))  
gdat <- as.matrix(gdat)  
cdat <- as.data.frame(cdat)  
  
# Add patient IDs if needed  
if (is.null(rownames(gdat))) {
```

```

rownames(gdat) <- paste0("Patient_", seq_len(nrow(gdat)))
}
if (is.null(rownames(cdat))) {
  rownames(cdat) <- rownames(gdat)
}

# Remove genes with zero variance.
# For this already log-transformed expression dataset, we keep the raw matrix
# for clustering/testing/network starter code. For dimension reduction, we
# center each gene but do not scale by the standard deviation.
#
# The one exception to the no-scaling rule is the bicluster heatmap below,
# where we scale the whole matrix only for visualization so gene-wise patterns
# are easier to see.
gene_sd <- apply(gdat, 2, sd, na.rm = TRUE)
gdat <- gdat[, gene_sd > 0, drop = FALSE]

# Raw analysis matrix used for clustering/testing/network starter code.
X <- gdat

# Centered-but-not-scaled matrix used for dimension reduction.
X_dr_centered <- scale(gdat, center = TRUE, scale = FALSE)

# Quick dimensions.
dim(gdat)

## [1] 445 353
dim(cdat)

## [1] 445 6
head(cdat)

```

```

##           Subtype ER-Status PR-Status HER2-Status Node Metastasis
## TCGA-A1-AOSD Luminal A Positive Positive Negative 0 0
## TCGA-A1-AOSE Luminal A Positive Positive Negative 0 0
## TCGA-A1-AOSH Luminal A Negative Positive Negative 0 0
## TCGA-A1-AOSJ Luminal A Positive Positive Negative 1 0
## TCGA-A1-AOSK Basal-like Negative Negative Negative 0 0
## TCGA-A1-AOSM Luminal B Positive Negative Positive 0 0

```

Explore clinical variables

```

clinical_vars <- c("Subtype", "ER", "PR", "HER2", "Node", "Metastasis")
clinical_vars <- intersect(clinical_vars, colnames(cdat))

for (v in clinical_vars) {
  cat("\n", v, "\n", sep = "")
  print(table(cdat[[v]], useNA = "ifany"))
}

##
## Subtype
##

```

```
##      Basal-like HER2-enriched      Luminal A      Luminal B      Normal-like
##              79              53              200              106              7
##
## Node
##
##      0      1      2      3 <NA>
## 221 146   54   23    1
##
## Metastasis
##
##      0      1 <NA>
## 427   11    7

# Example cross-tabulation.
if (all(c("ER", "PR") %in% colnames(cdat))) {
  table(ER = cdat$ER, PR = cdat$PR, useNA = "ifany")
}
```

Helper functions

```
label_colors <- function(labels) {
  labels <- factor(labels)
  pal <- grDevices::hcl.colors(nlevels(labels), palette = "Dark 3")
  out <- pal[as.integer(labels)]
  names(out) <- as.character(labels)
  out
}

plot_embedding <- function(df, x, y, color_var, label_var = NULL, title = NULL) {
  if (!pkg_available("ggplot2")) {
    plot(df[[x]], df[[y]], pch = 16, col = label_colors(df[[color_var]]),
         xlab = x, ylab = y, main = title)
    legend("topright", legend = levels(factor(df[[color_var]])),
          col = unique(label_colors(df[[color_var]])), pch = 16, cex = 0.8)
    return(invisible(NULL))
  }

  p <- ggplot(df, aes(x = .data[[x]], y = .data[[y]], color = .data[[color_var]])) +
    geom_point(size = 2, alpha = 0.85) +
    labs(title = title, x = x, y = y, color = color_var) +
    theme_minimal()

  if (!is.null(label_var)) {
    p <- p + geom_text(aes(label = .data[[label_var]]), size = 2.5, show.legend = FALSE)
  }

  p
}

count_edges <- function(adj) {
  adj <- as.matrix(adj)
  adj[lower.tri(adj, diag = TRUE)] <- 0
  sum(adj != 0, na.rm = TRUE)
}
```

```
safe_ttest_p <- function(x, group) {
  keep <- !is.na(x) & !is.na(group)
  x <- x[keep]
  group <- droplevels(factor(group[keep]))
  if (nlevels(group) != 2) return(NA_real_)
  if (any(table(group) < 2)) return(NA_real_)
  tryCatch(t.test(x ~ group)$p.value, error = function(e) NA_real_)
}
```

Starter code for Problem 1: Dimension reduction

PCA

```
# PCA on the centered-but-not-scaled expression matrix.
# This is equivalent to prcomp(gdat, center = TRUE, scale. = FALSE), but using
# X_dr_centered makes the preprocessing step explicit for students.
pca_fit <- prcomp(X_dr_centered, center = FALSE, scale. = FALSE)

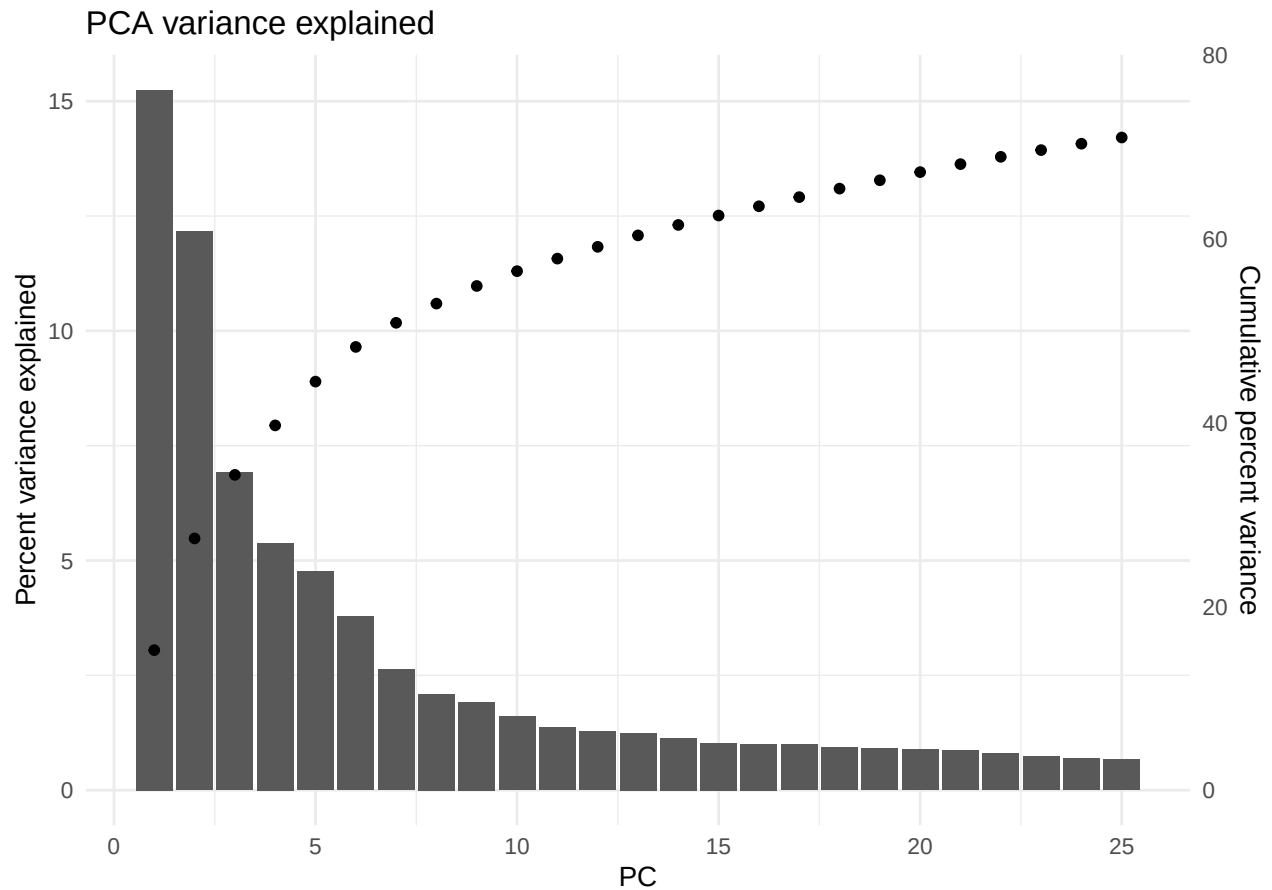
pca_var <- pca_fit$sdev^2
pca_prop <- pca_var / sum(pca_var)

pca_var_df <- data.frame(
  PC = seq_along(pca_prop),
  PercentVariance = 100 * pca_prop,
  CumulativeVariance = 100 * cumsum(pca_prop)
)

head(pca_var_df, 10)
```

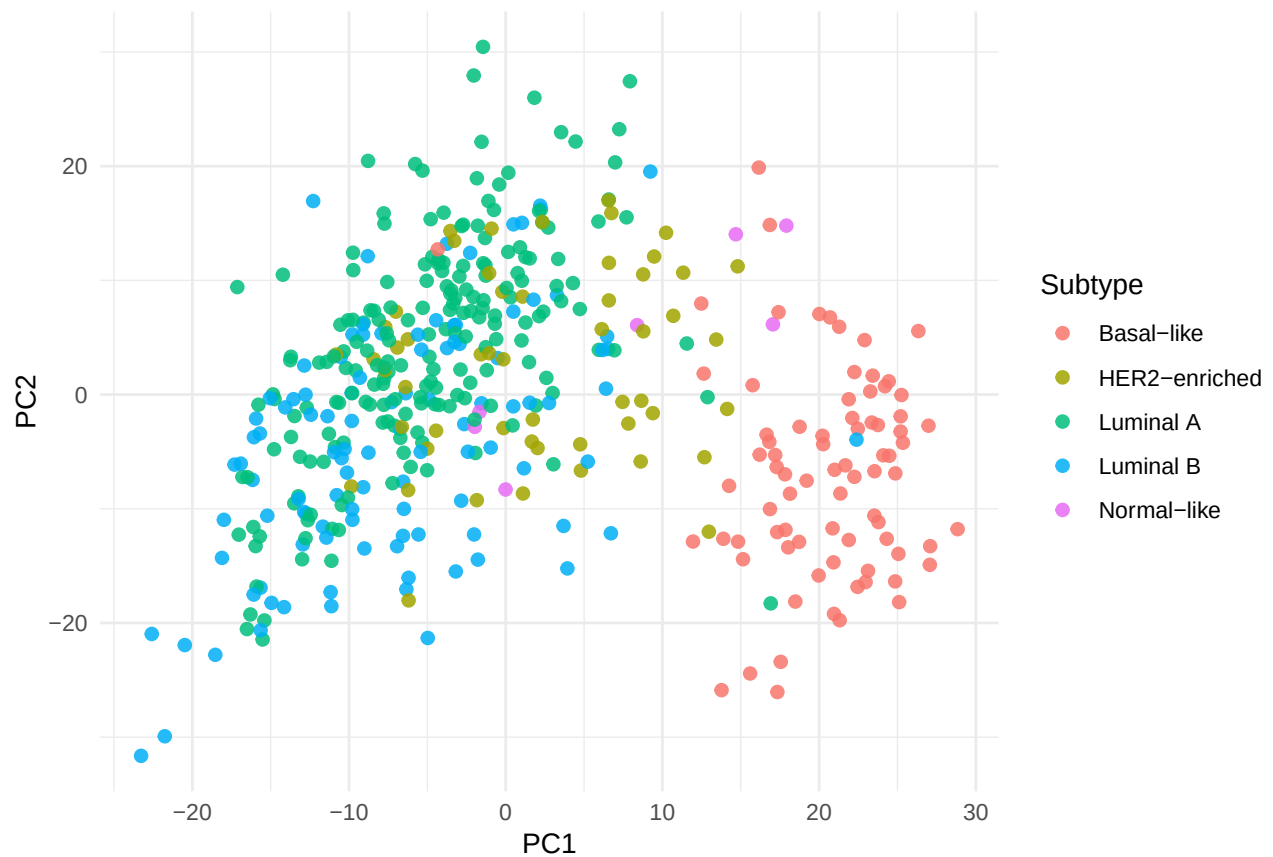
##	PC	PercentVariance	CumulativeVariance
## 1	1	15.246613	15.24661
## 2	2	12.158441	27.40505
## 3	3	6.920932	34.32599
## 4	4	5.383428	39.70941
## 5	5	4.766763	44.47618
## 6	6	3.776380	48.25256
## 7	7	2.624594	50.87715
## 8	8	2.091619	52.96877
## 9	9	1.918793	54.88756
## 10	10	1.610195	56.49776

```
ggplot(pca_var_df[1:25, ], aes(x = PC, y = PercentVariance)) +
  geom_col() +
  geom_point(aes(y = CumulativeVariance / 5)) +
  scale_y_continuous(
    name = "Percent variance explained",
    sec.axis = sec_axis(~ . * 5, name = "Cumulative percent variance")
  ) +
  labs(title = "PCA variance explained") +
  theme_minimal()
```

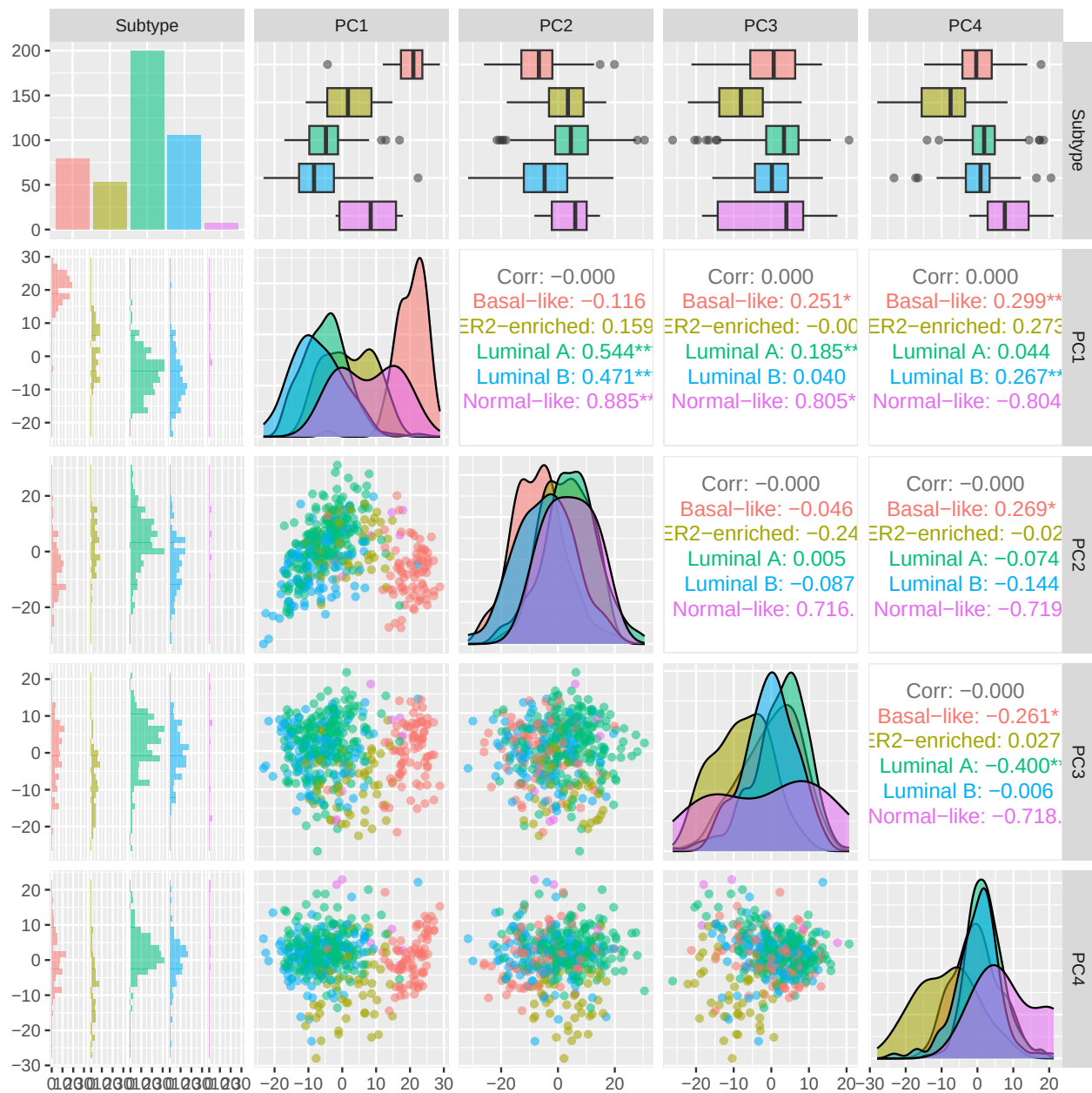


```
pca_df <- data.frame(  
  Patient = rownames(gdat),  
  Subtype = cdat$Subtype,  
  PC1 = pca_fit$x[, 1],  
  PC2 = pca_fit$x[, 2],  
  PC3 = pca_fit$x[, 3],  
  PC4 = pca_fit$x[, 4]  
)  
  
plot_embedding(  
  pca_df,  
  x = "PC1",  
  y = "PC2",  
  color_var = "Subtype",  
  title = "BRCA expression data: PCA colored by subtype"  
)
```

BRCA expression data: PCA colored by subtype



```
# Inspect several PCs together.
# Explicitly print the ggpairs object so the plot appears when knitting.
if (load_pkg("GGally")) {
  p_pca_pairs <- GGally::ggpairs(
    pca_df[, c("Subtype", "PC1", "PC2", "PC3", "PC4")],
    mapping = ggplot2::aes(color = Subtype, alpha = 0.7)
  )
  print(p_pca_pairs)
} else {
  message("Install the GGally package to display the PCA pairs plot.")
}
```

MDS

```
# Classical MDS using centered-but-not-scaled data.
# We show two non-Euclidean distances so students can compare the geometry.
D_mds_max <- dist(X_dr_centered, method = "maximum")
mds_max <- cmdscale(D_mds_max, k = 2)

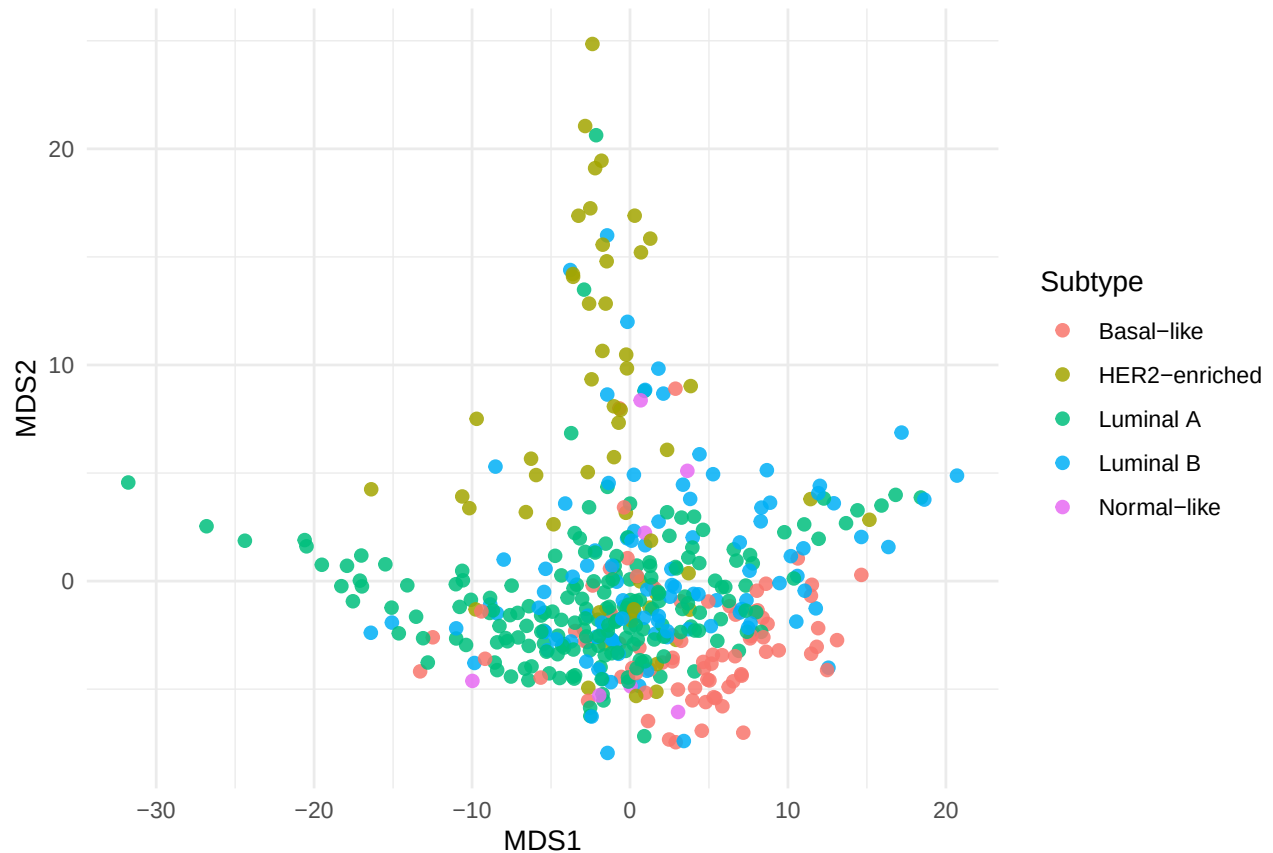
mds_max_df <- data.frame(
  Patient = rownames(gdat),
  Subtype = cdat$Subtype,
  MDS1 = mds_max[, 1],
  MDS2 = mds_max[, 2]
)
```

```

p_mds_max <- plot_embedding(
  mds_max_df,
  x = "MDS1",
  y = "MDS2",
  color_var = "Subtype",
  title = "Classical MDS using maximum distance"
)
print(p_mds_max)

```

Classical MDS using maximum distance



```

# A second MDS view using Manhattan distance.
D_mds_man <- dist(X_dr_centered, method = "manhattan")
mds_man <- cmdscale(D_mds_man, k = 2)

mds_man_df <- data.frame(
  Patient = rownames(gdat),
  Subtype = cdat$Subtype,
  MDS1 = mds_man[, 1],
  MDS2 = mds_man[, 2]
)

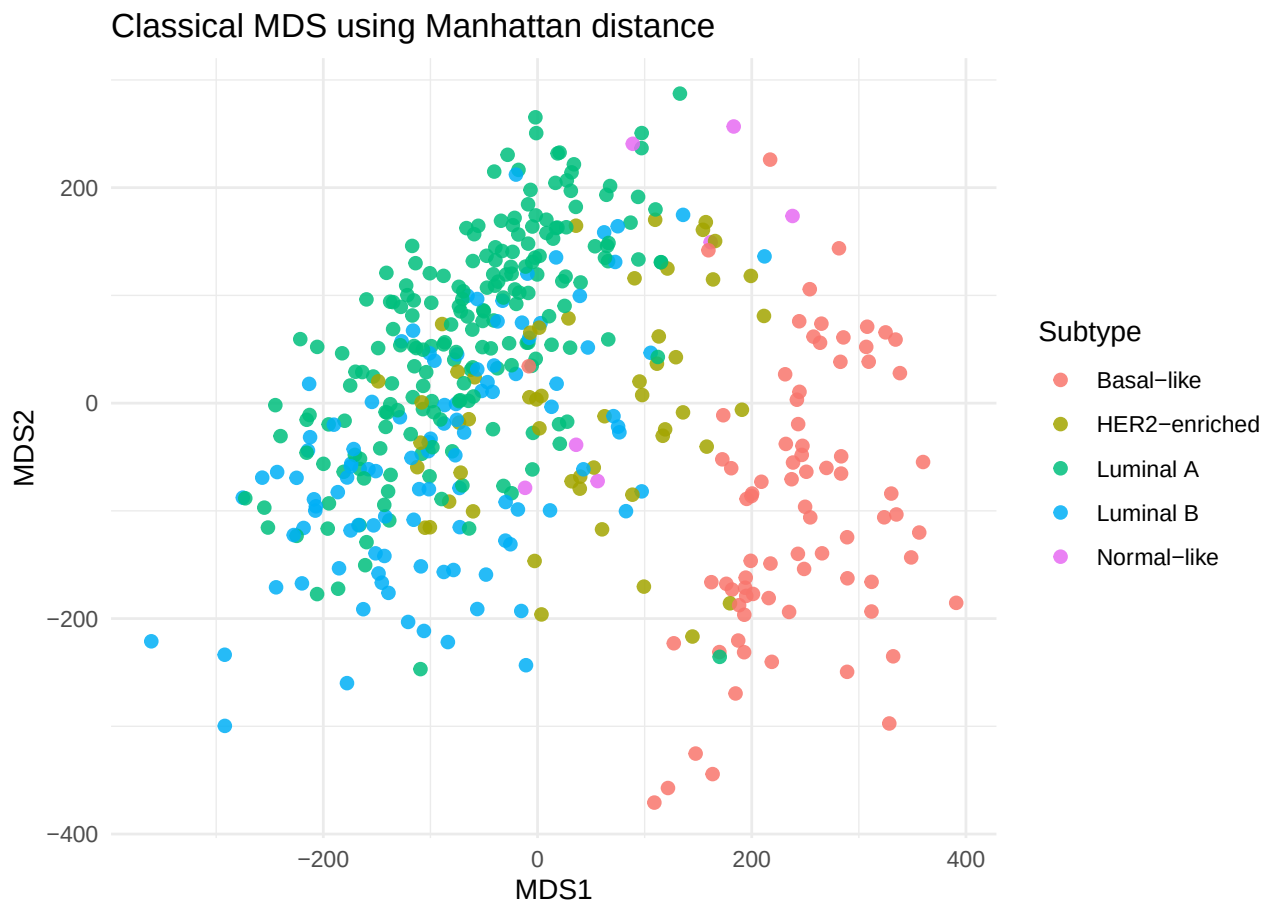
p_mds_man <- plot_embedding(
  mds_man_df,
  x = "MDS1",
  y = "MDS2",
  color_var = "Subtype",

```

```

    title = "Classical MDS using Manhattan distance"
  )
  print(p_mds_man)

```



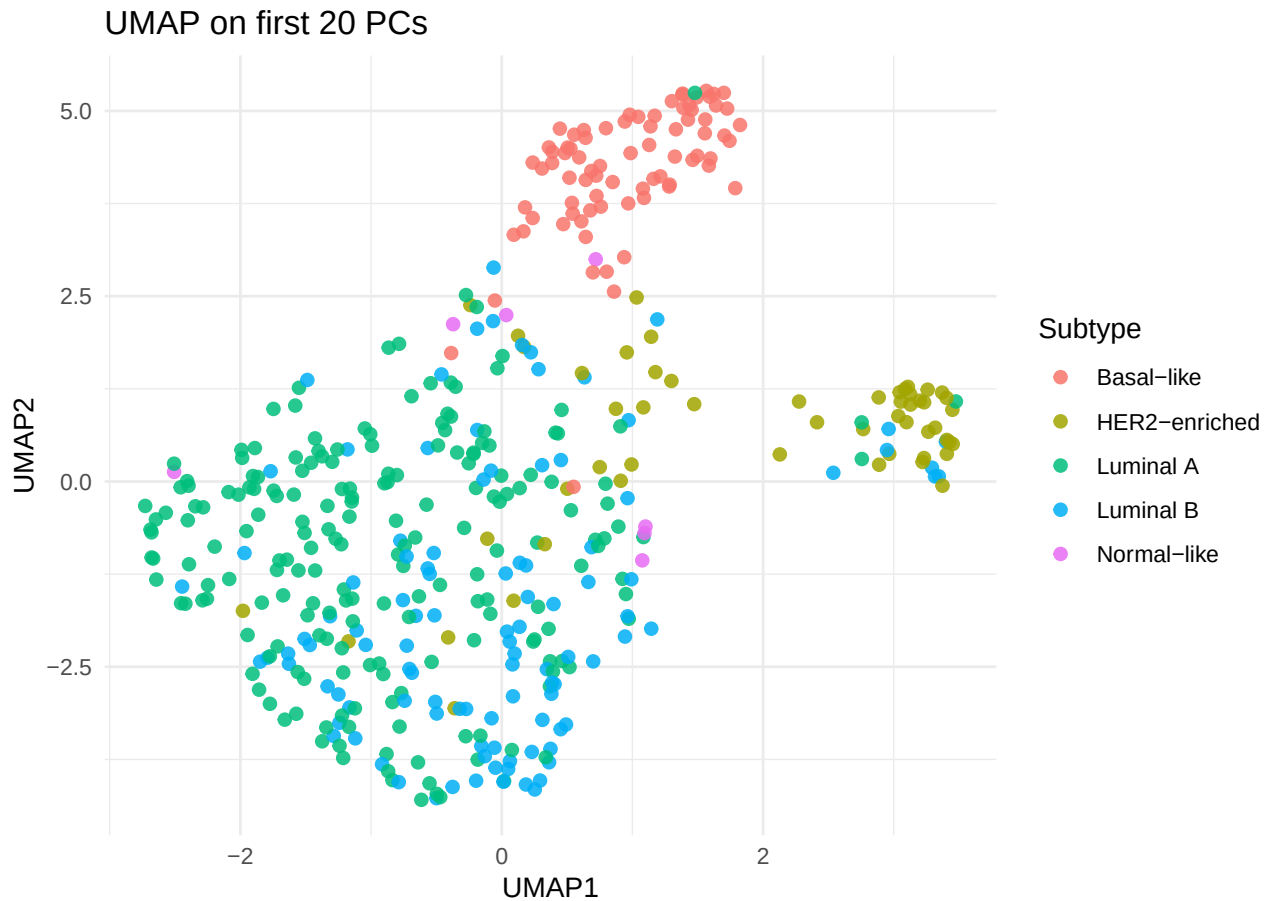
UMAP and t-SNE

```

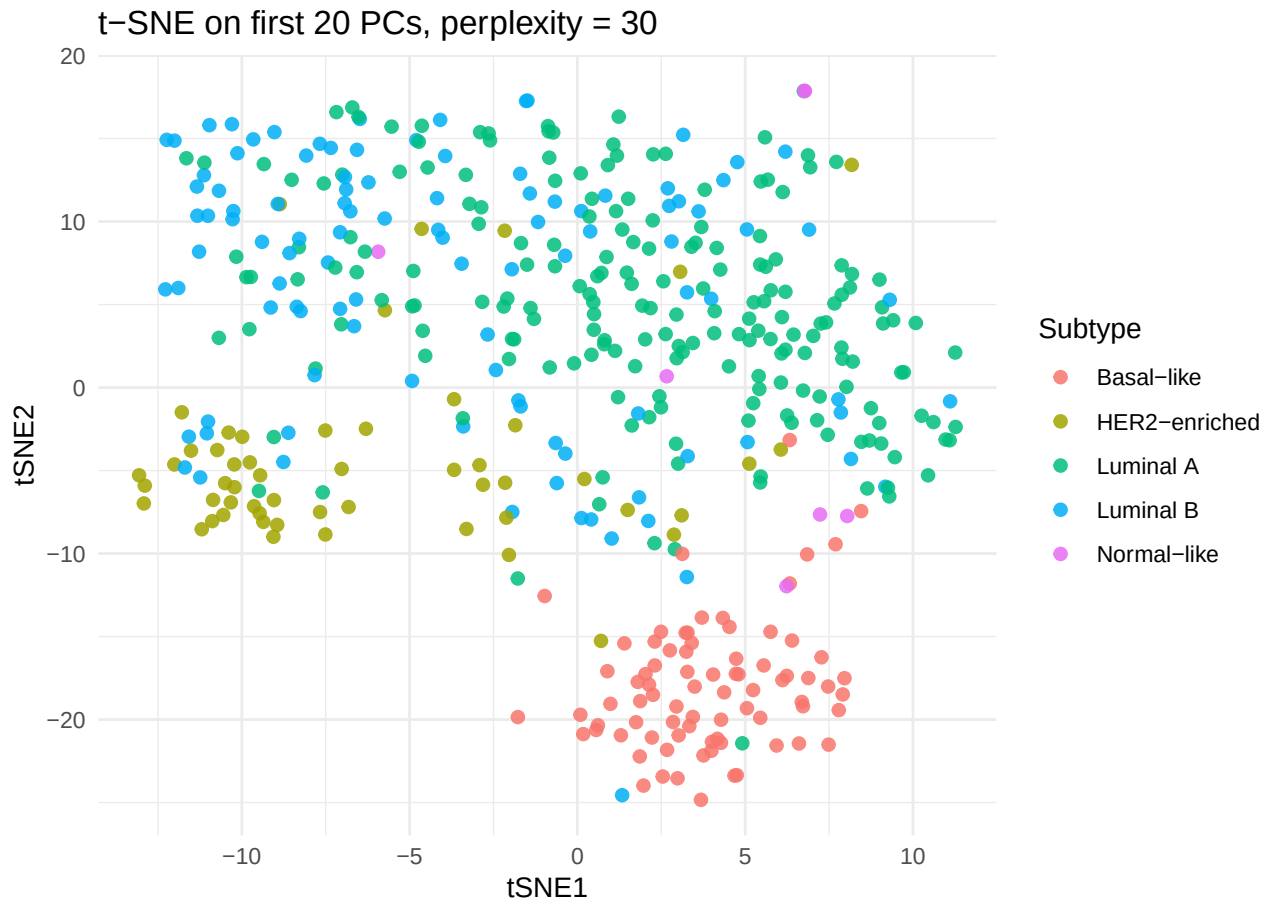
# UMAP can be sensitive to tuning parameters.
# We use the first 20 PCs as input to reduce noise and speed up the algorithm.
X_dr <- pca_fit$x[, 1:20, drop = FALSE]

if (load_pkg("umap")) {
  set.seed(2026)
  umap_fit <- umap::umap(X_dr)
  umap_df <- data.frame(
    Subtype = cdat$Subtype,
    UMAP1 = umap_fit$layout[, 1],
    UMAP2 = umap_fit$layout[, 2]
  )
  print(plot_embedding(
    umap_df, "UMAP1", "UMAP2", "Subtype",
    title = "UMAP on first 20 PCs"
  ))
}

```



```
# t-SNE is also sensitive to tuning parameters.
# Perplexity must be smaller than the number of samples.
if (load_pkg("Rtsne")) {
  set.seed(2026)
  tsne_perplexity <- min(30, floor((nrow(X_dr) - 1) / 3))
  tsne_fit <- Rtsne::Rtsne(
    X_dr,
    perplexity = tsne_perplexity,
    check_duplicates = FALSE,
    verbose = FALSE
  )
  tsne_df <- data.frame(
    Subtype = cdat$Subtype,
    tSNE1 = tsne_fit$Y[, 1],
    tSNE2 = tsne_fit$Y[, 2]
  )
  print(plot_embedding(
    tsne_df, "tSNE1", "tSNE2", "Subtype",
    title = paste("t-SNE on first 20 PCs, perplexity =", tsne_perplexity)
  ))
}
```



Starter code for Problem 2: Clustering

Heatmap and biclustering-style exploration

```
# Bicluster-style heatmap.
# For this display only, scale the whole gene-expression matrix so genes with
# different expression ranges can be compared visually. Do not use this scaled
# matrix as the main analysis matrix elsewhere in the lab.
aa <- grep("grey", colors())
bb <- grep("green", colors())
cc <- grep("red", colors())
gcol2 <- colors()[c(aa[1:2], bb[1:25], cc[1:50])]

X_heat <- scale(gdat)

subtype <- factor(cdat$Subtype)
subtype_palette <- setNames(rainbow(nlevels(subtype)), levels(subtype))
subtype_colors <- subtype_palette[as.character(subtype)]

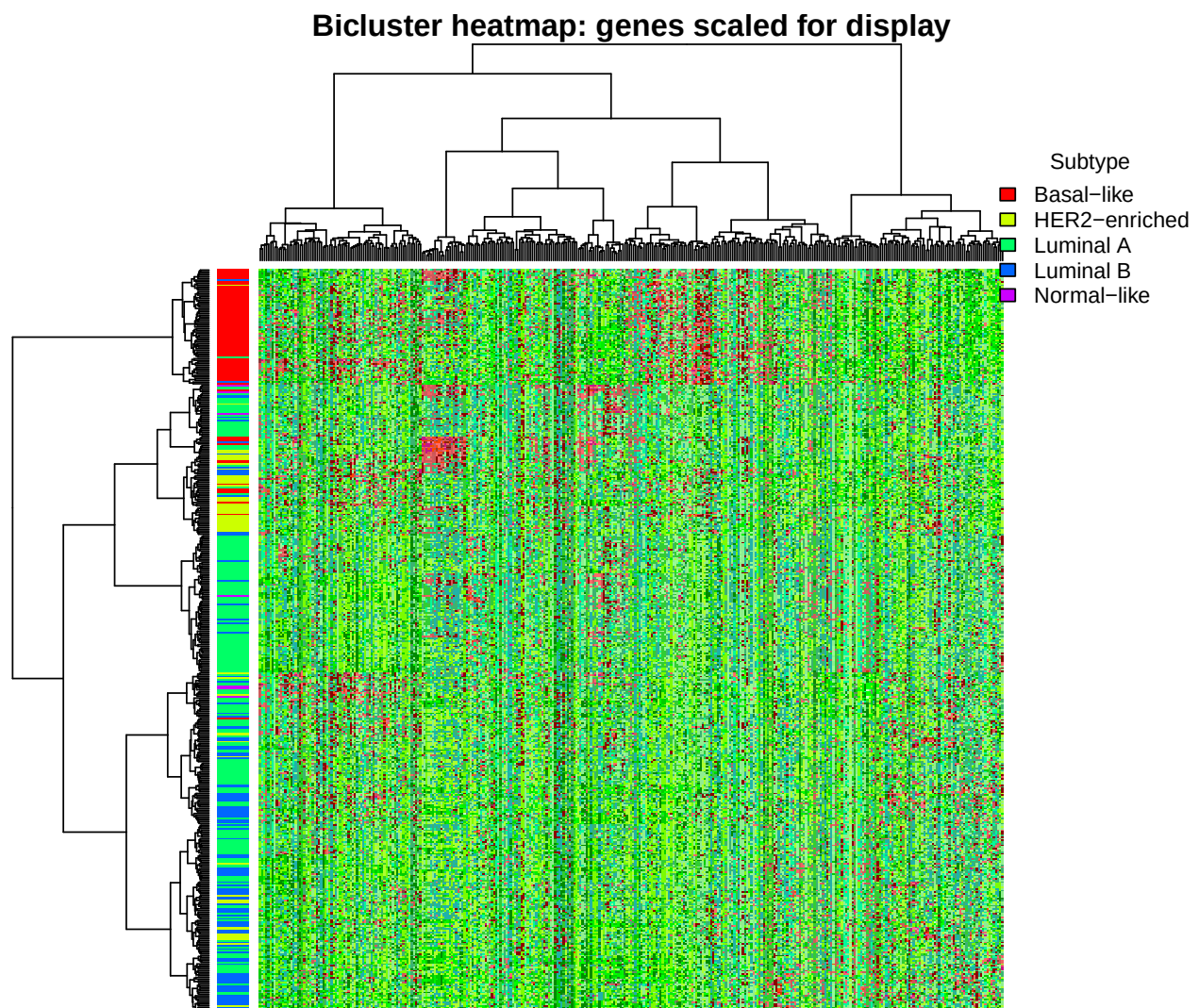
heatmap(
  X_heat,
  col = gcol2,
  scale = "none",
  hclustfun = function(x) hclust(x, method = "ward.D"),
```

```

labRow = FALSE,
labCol = FALSE,
RowSideColors = subtype_colors,
main = "Biclust heatmap: genes scaled for display"
)

legend(
  "topright",
  legend = names(subtype_palette),
  fill = subtype_palette,
  title = "Subtype",
  cex = 0.8,
  bty = "n"
)

```



K-means clustering

```

# Starter: run K-means for one value of K.
# Students: compare several K values and decide how to evaluate them.

```

```

set.seed(2026)
K <- 5
km_fit <- kmeans(X, centers = K, nstart = 50)

# Compare clusters to subtype as an external interpretation aid.
# Do not use subtype to fit the clusters.
table(KMeansCluster = km_fit$cluster, Subtype = cdat$Subtype)

##              Subtype
## KMeansCluster Basal-like HER2-enriched Luminal A Luminal B Normal-like
##              1           5              8          44          12           4
##              2           0              2          40          60           1
##              3          74              5           1           1           1
##              4           0             31           3           7           0
##              5           0              7          112          26           1

```

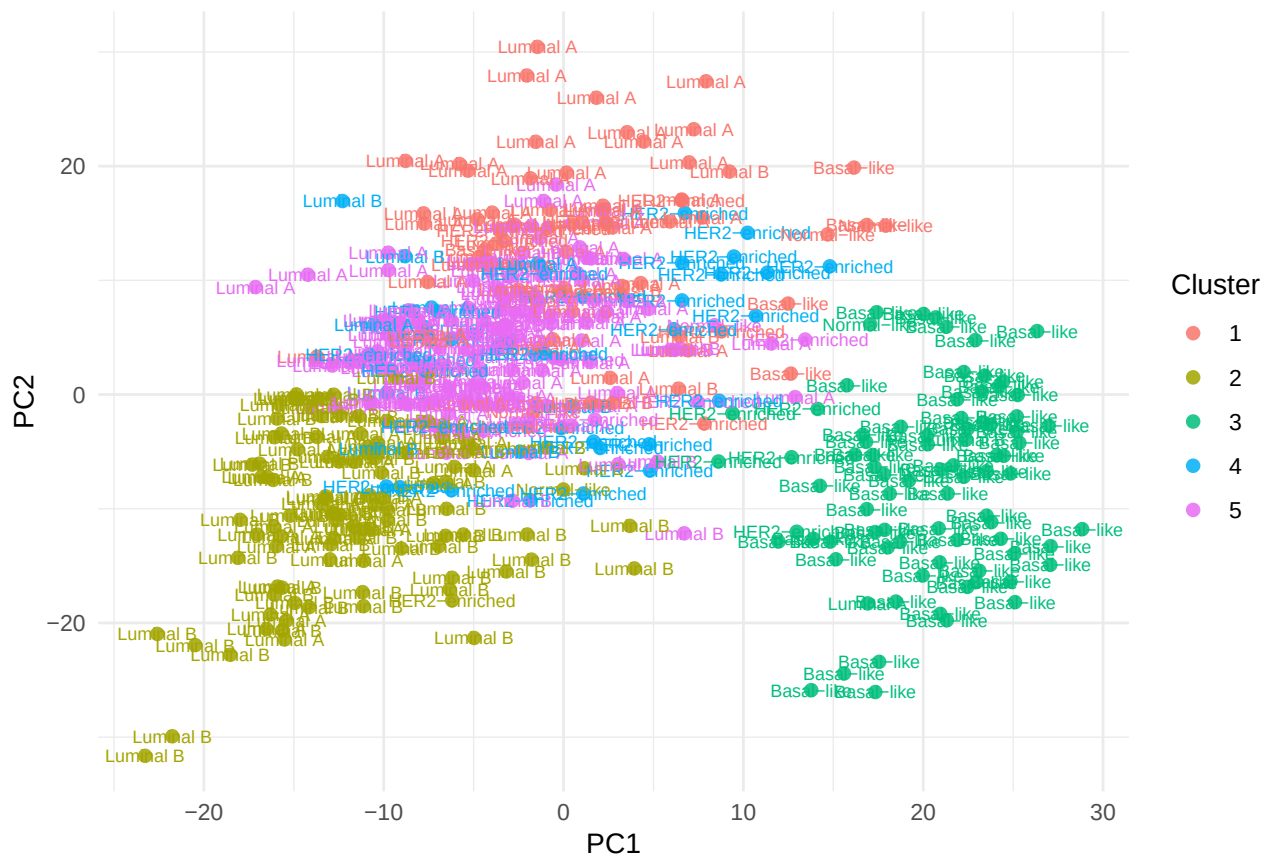
```

km_pca_df <- data.frame(
  Subtype = cdat$Subtype,
  Cluster = factor(km_fit$cluster),
  PC1 = pca_fit$x[, 1],
  PC2 = pca_fit$x[, 2]
)

plot_embedding(
  km_pca_df,
  x = "PC1",
  y = "PC2",
  color_var = "Cluster",
  label_var = "Subtype",
  title = "K-means clusters visualized on first two PCs"
)

```

K-means clusters visualized on first two PCs

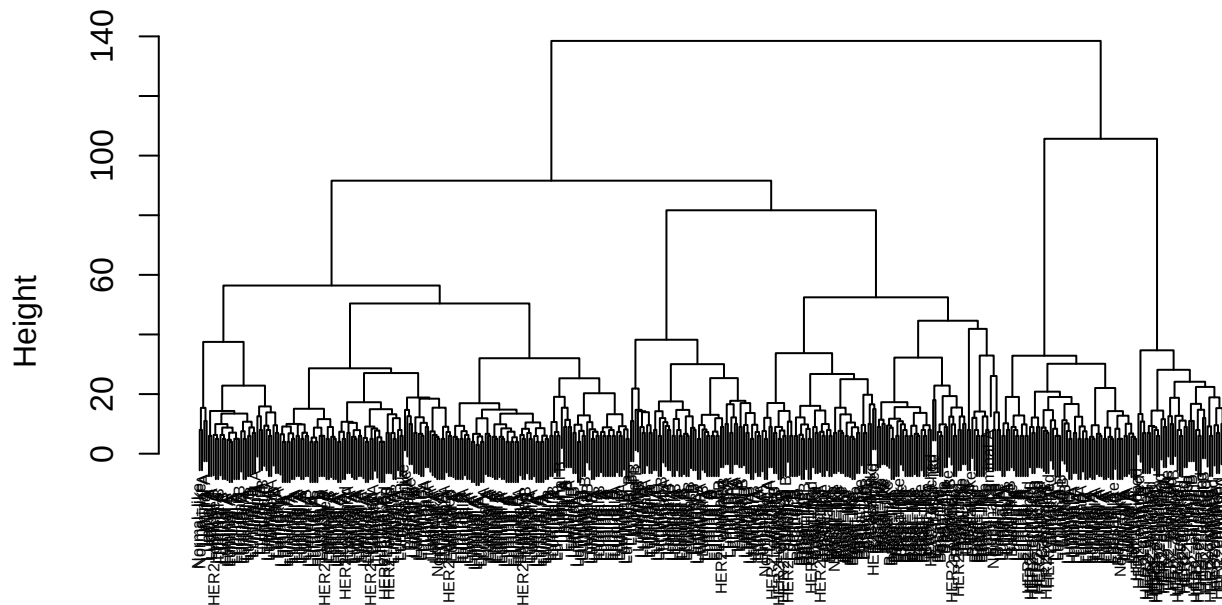


Hierarchical clustering

```
# Starter: hierarchical clustering on the raw expression matrix.
# Students: try other distances and linkage methods.
D_hc <- dist(X, method = "maximum")
hc_fit <- hclust(D_hc, method = "ward.D2")

plot(hc_fit, labels = cdat$Subtype, cex = 0.5,
     main = "Hierarchical clustering dendrogram")
```


Hierarchical clustering dendrogram



D_hc
hclust (*, "ward.D2")

```
hc_clusters <- cutree(hc_fit, k = 5)
table(HierCluster = hc_clusters, Subtype = cdat$Subtype)
```

```
##          Subtype
## HierCluster Basal-like HER2-enriched Luminal A Luminal B Normal-like
##          1           3              8        126        47          3
##          2           4              7         36         11          1
##          3           0              1         26         30          0
##          4          71             10          9         11          3
##          5           1             27          3          7          0
```

Visualize the flat hierarchical clustering assignments in PCA space.
This mirrors the K-means PCA visualization above.

```
hc_pca_df <- data.frame(
  Subtype = cdat$Subtype,
  Cluster = factor(hc_clusters),
  PC1 = pca_fit$x[, 1],
  PC2 = pca_fit$x[, 2]
)
```

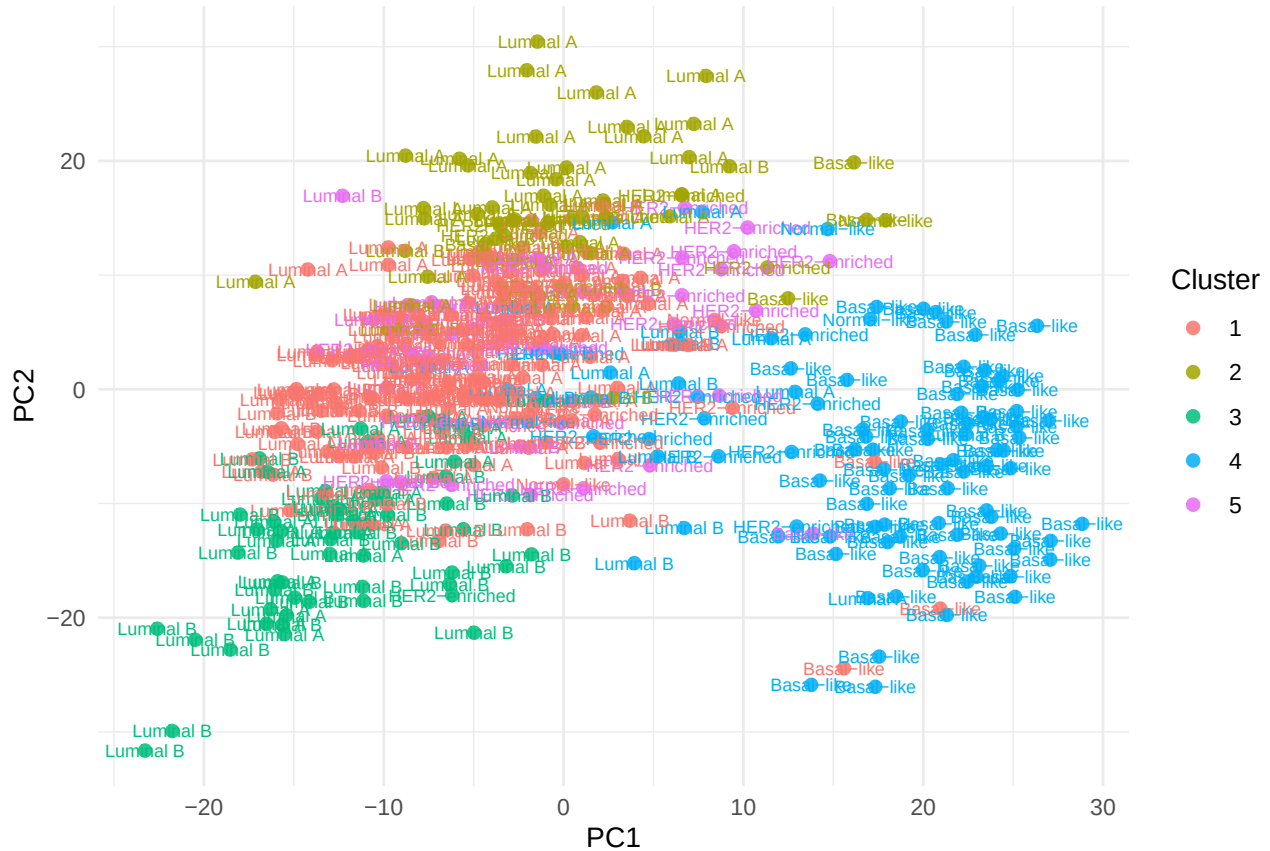
```
p_hc_pca <- plot_embedding(
  hc_pca_df,
  x = "PC1",
  y = "PC2",
  color_var = "Cluster",
  label_var = "Subtype",
```

```

title = "Hierarchical clusters visualized on first two PCs"
)
print(p_hc_pca)

```

Hierarchical clusters visualized on first two PCs



```

# Starter template: compare alternative linkages or distances.
D_hc_alt <- dist(X, method = "manhattan")
hc_alt <- hclust(D_hc_alt, method = "average")
plot(hc_alt, labels = cdat$Subtype, cex = 0.5)

```

Starter code for Problem 3: Multiple comparisons

Two-group testing helper

```

two_group_gene_tests <- function(X, group) {
  group <- droplevels(factor(group))
  pvals <- apply(X, 2, safe_ttest_p, group = group)

  data.frame(
    Gene = colnames(X),
    PValue = pvals,
    Bonferroni = p.adjust(pvals, method = "bonferroni"),
    Holm = p.adjust(pvals, method = "holm"),
    BH = p.adjust(pvals, method = "BH"),
    BY = p.adjust(pvals, method = "BY"),
  )
}

```

```

    row.names = NULL
  )
}

summarize_rejections <- function(test_df, alpha = 0.10) {
  data.frame(
    Method = c("Unadjusted", "Bonferroni", "Holm", "BH", "BY"),
    Rejections = c(
      sum(test_df$PValue < alpha, na.rm = TRUE),
      sum(test_df$Bonferroni < alpha, na.rm = TRUE),
      sum(test_df$Holm < alpha, na.rm = TRUE),
      sum(test_df$BH < alpha, na.rm = TRUE),
      sum(test_df$BY < alpha, na.rm = TRUE)
    )
  )
}

```

Example: ER status

```

# Starter example for ER status.
# Students: repeat and adapt this for PR, HER2, and metastasis.
er_keep <- cdat$ER %in% c("Positive", "Negative")
X_er <- gdat[er_keep, , drop = FALSE]
er_group <- droplevels(factor(cdat$ER[er_keep]))

er_tests <- two_group_gene_tests(X_er, er_group)
er_tests <- er_tests[order(er_tests$PValue), ]

head(er_tests, 10)

```

##	Gene	PValue	Bonferroni	Holm	BH	BY
## 131	GATA3	6.134126e-40	2.165346e-37	2.165346e-37	2.165346e-37	1.395587e-36
## 155	IL6ST	9.374727e-39	3.309279e-36	3.299904e-36	1.654639e-36	1.066431e-35
## 220	MYST4	1.006078e-31	3.551456e-29	3.531334e-29	1.183819e-29	7.629829e-29
## 25	BCL2	2.251878e-28	7.949130e-26	7.881574e-26	1.987283e-26	1.280823e-25
## 124	FOXA1	4.135372e-28	1.459786e-25	1.443245e-25	2.919573e-26	1.881694e-25
## 240	NUMA1	6.276480e-26	2.215597e-23	2.184215e-23	3.692662e-24	2.379958e-23
## 41	CACNA1D	3.261112e-25	1.151173e-22	1.131606e-22	1.644532e-23	1.059917e-22
## 54	CCND1	2.912363e-24	1.028064e-21	1.007678e-21	1.285080e-22	8.282471e-22
## 282	RET	7.715962e-24	2.723734e-21	2.662007e-21	3.026372e-22	1.950527e-21
## 214	MYB	1.232211e-23	4.349706e-21	4.238807e-21	4.349706e-22	2.803429e-21

```

summarize_rejections(er_tests, alpha = 0.10)

```

##	Method	Rejections
## 1	Unadjusted	263
## 2	Bonferroni	168
## 3	Holm	177
## 4	BH	259
## 5	BY	217

```

alpha <- 0.10
m <- nrow(er_tests)
ordered_p <- sort(er_tests$PValue)

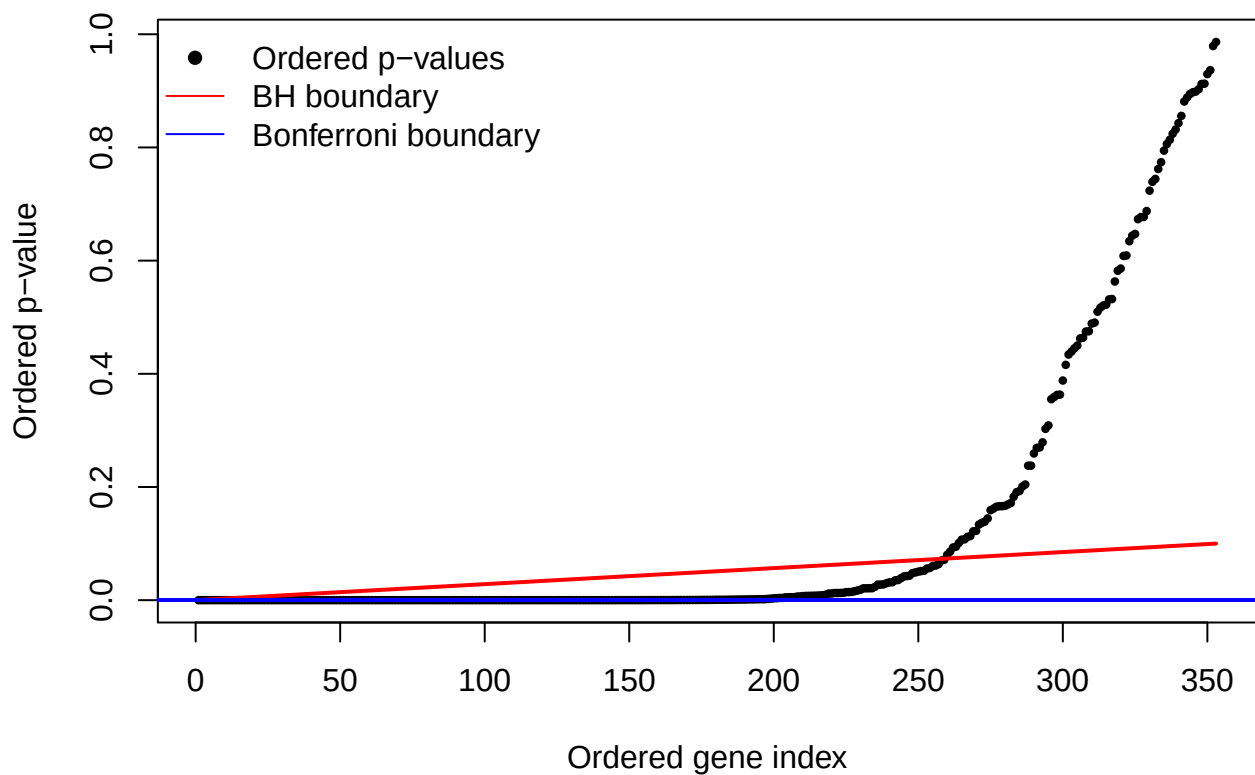
```

```

plot(
  ordered_p,
  pch = 16,
  cex = 0.6,
  xlab = "Ordered gene index",
  ylab = "Ordered p-value",
  main = "ER status: ordered p-values"
)
lines(seq_len(m), alpha * seq_len(m) / m, col = "red", lwd = 2)
abline(h = alpha / m, col = "blue", lwd = 2)
legend(
  "topleft",
  legend = c("Ordered p-values", "BH boundary", "Bonferroni boundary"),
  pch = c(16, NA, NA),
  lty = c(NA, 1, 1),
  col = c("black", "red", "blue"),
  bty = "n"
)

```

ER status: ordered p-values



```

# Starter template for another clinical response.
# Replace RESPONSE with PR, HER2, or Metastasis.
response <- cdat$PR
keep <- response %in% c("Positive", "Negative")

test_df <- two_group_gene_tests(
  X = gdat[keep, , drop = FALSE],
  group = response[keep]
)

```

```
)

test_df <- test_df[order(test_df$PValue), ]
head(test_df, 10)
summarize_rejections(test_df, alpha = 0.10)
```

Starter code for Problem 4: Graphical models

Correlation network starter

```
# Use a smaller set of genes for readable networks.
# Students: try different gene subsets, such as top variable genes or top genes from testing.
top_var_genes <- names(sort(apply(gdat, 2, var), decreasing = TRUE))
net_genes <- top_var_genes[1:40]
X_net <- X[, net_genes]

cor_mat <- cor(X_net)
cor_threshold <- 0.50
adj_cor <- abs(cor_mat) > cor_threshold
diag(adj_cor) <- FALSE

count_edges(adj_cor)

## [1] 25

if (load_pkg("igraph")) {
  g_cor <- graph_from_adjacency_matrix(adj_cor, mode = "undirected", diag = FALSE)
  plot(
    g_cor,
    vertex.label = colnames(X_net),
    vertex.size = 5,
    vertex.label.cex = 0.7,
    layout = layout_with_fr(g_cor),
    main = paste("Correlation network, |r| >", cor_threshold)
  )
}
```

Correlation network, $|r| > 0.5$

